

ORGANISATION DU TRAVAIL

ERP CFA

Méthode de répartition, branches Git, merge et planning de développement

Version de travail - Juin 2026

Objectif : organiser le développement de l'ERP CFA entre deux développeurs juniors, en travaillant par petites features, avec des branches séparées, des merges réguliers et une validation commune avant intégration.

1. Principe général

La méthode recommandée consiste à faire travailler chaque développeur sur une feature courte et indépendante, puis à intégrer régulièrement le travail dans une branche commune. L'objectif n'est pas de coder chacun de son côté pendant plusieurs semaines, mais de livrer des blocs fonctionnels testables.

Règle principale : une feature doit être petite, claire, testable et mergée rapidement. Une feature trop grosse augmente les conflits, les bugs et les écarts entre le backend, l'interface et les règles métier.

2. Organisation Git recommandée

Le projet doit utiliser deux branches principales :

- main : branche stable, utilisée uniquement pour les versions propres ou démontrables.
- develop : branche commune de développement, utilisée pour intégrer les features validées.

Chaque fonctionnalité doit être développée dans une branche dédiée :

Branche	Rôle
feature/auth-roles	Connexion, utilisateurs, rôles et permissions.
feature/candidates	Module candidats : création, liste, filtres, fiche, statuts.
feature/companies	Module entreprises : fiche entreprise, contacts, tuteurs.
feature/needs	Besoins entreprises : postes ouverts, statuts, formation visée.
feature/matching	Rattachement candidat / besoin, suivi CV, entretien, retour entreprise.
feature/admission	Dossier admission, checklist, pièces obligatoires, validation.
feature/documents	Documents, upload, statuts, conformité, rattachement dossier.
feature/contracts	Contrats, conventions, statuts, documents signés.
feature/opco	Dossiers OPCO, dépôt, rejet, correction, suivi blocage.
feature/tasks-alerts	Tâches, relances, alertes et priorités utilisateur.
feature/dashboard	Tableau de bord simple, indicateurs et listes filtrées.

3. Méthode de travail par feature

Chaque feature doit suivre le même cycle de travail :

1. Créer une branche feature depuis develop.

2. Développer une fonctionnalité courte et complète.
3. Tester localement la feature.
4. Faire un commit clair.
5. Pousser la branche sur GitHub.
6. Créer une Pull Request vers develop.
7. Faire relire par le coéquipier.
8. Corriger les remarques si nécessaire.
9. Merger dans develop.
10. Tester rapidement develop après le merge.

4. Commandes Git utiles

```
git checkout develop
git pull origin develop
git checkout -b feature/candidates
```

```
# après développement
git status
git add .
git commit -m "Add candidates module"
git push origin feature/candidates
```

5. Répartition conseillée entre les deux développeurs

La répartition doit rester souple, mais chaque développeur peut être responsable d'un type de travail. L'idée est de séparer les responsabilités sans bloquer l'autre.

Sujet	Développeur 1	Développeur 2
Socle initial	Auth, rôles, permissions, règles métier	Layout, navigation, écrans de base
Candidats	Modèle, statuts, validations, historique	Liste, formulaire, fiche, filtres
Entreprises	Relations entreprise, contacts, tuteurs	Interface fiche entreprise et tableau
Besoins	Statuts, relations, règles de clôture	Écrans besoins, filtres, affichage postes
Matching	Modèle, règles, statuts	Interface de proposition et suivi retours
Admission	Checklist, validation, blocages	Interface dossier et pièces manquantes
Documents	Rattachement, règles de version	Upload, affichage, statuts conformité
Contrats	Statuts, règles de signature	Écran contrat, documents, suivi visuel
OPCO	Règles rejet/correction, statuts	Tableau des dossiers bloqués
Dashboard	Calculs KPI, requêtes	Design, cartes, filtres, listes cliquables

6. Ordre de développement conseillé

Les features doivent être développées dans un ordre logique, car certains modules dépendent des autres.

Sprint	Features principales	Objectif
Sprint 1	auth-roles, layout-navigation	Connexion, menu, structure générale.
Sprint 2	candidates, companies	Créer les candidats et entreprises.
Sprint 3	needs, matching	Créer les besoins et proposer des candidats.
Sprint 4	documents, admission	Gérer les pièces, checklist et validation.

Sprint 5	contracts, opco	Suivre contrat, dépôt OPCO, rejet et correction.
Sprint 6	tasks-alerts, history, dashboard	Piloter les actions, alertes et indicateurs.

7. Backlog P0 recommandé

Pour éviter de se disperser, la première version doit couvrir uniquement le parcours indispensable : candidat -> entreprise -> besoin -> matching -> admission -> contrat -> OPCO -> dashboard.

ID	Feature	Résultat attendu
P0-01	Authentification et rôles	Un utilisateur se connecte et voit seulement les modules autorisés.
P0-02	Module candidats	Créer, modifier, filtrer et suivre un candidat.
P0-03	Module entreprises	Créer une entreprise avec contacts et tuteurs.
P0-04	Besoins entreprises	Créer un poste ouvert et suivre son statut.
P0-05	Matching	Proposer un candidat à un besoin et suivre le retour.
P0-06	Admission	Checklist pièces, conformité, validation ou refus.
P0-07	Documents	Upload, rattachement, statut, historique simple.
P0-08	Contrats	Créer un dossier contrat et suivre signature/statut.
P0-09	OPCO	Suivre dépôt, rejet, correction et blocage.
P0-10	Tâches et alertes	Chaque utilisateur voit ses actions prioritaires.
P0-11	Historique	Les actions importantes sont journalisées.
P0-12	Dashboard simple	Direction visualise candidats, contrats, OPCO bloqués et tâches.

8. Règles anti-conflits

- Ne pas modifier les mêmes fichiers au même moment sans prévenir le coéquipier.
- Avant de coder une feature, valider ensemble les noms des tables, champs, statuts et relations.
- Faire des Pull Requests courtes : idéalement 1 à 3 jours de travail maximum.
- Mettre à jour sa branche avec develop avant de commencer une nouvelle journée de travail.
- Ne pas merger directement dans main.
- Tester develop après chaque merge important.
- Chaque feature doit être utilisable dans l'interface, même de manière simple.
- Éviter les fonctionnalités longues qui restent non mergées pendant plusieurs semaines.

9. Checklist avant Pull Request

- La feature fonctionne localement.
- Les migrations ou changements de base de données sont inclus.
- Les règles métier principales sont respectées.
- Les erreurs utilisateur sont compréhensibles.
- La page liste et la page détail fonctionnent si la feature a une interface.
- Les rôles et permissions sont vérifiés si la feature contient des données sensibles.
- Aucune donnée de test inutile n'est laissée dans le code.
- Le nom de commit est clair.
- La Pull Request explique ce qui a été ajouté et comment tester.

10. Que faire en cas de conflit de merge ?

1. Lire les fichiers en conflit et comprendre quelle partie appartient à quel développeur.
2. Ne pas accepter aveuglément toutes les modifications d'un seul côté.
3. Garder la version qui respecte le modèle de données validé ensemble.
4. Relancer l'application après résolution.
5. Tester la feature concernée et les features liées.
6. Faire un commit de résolution de conflit clair.
7. Informer le coéquipier des fichiers modifiés pendant la résolution.

11. Planning réaliste sur 8 semaines

Semaine	Objectif
S1	Setup projet, Git, auth, rôles, layout, navigation.
S2	Candidats, entreprises, premiers filtres et historiques simples.
S3	Besoins entreprises, matching, tâches commerciales.
S4	Admission, documents, checklist pièces et conformité.
S5	Contrats, statuts contrat, documents signés.
S6	OPCO, rejet, correction, dossiers bloqués.
S7	Dashboard, filtres, exports simples si nécessaire.
S8	Corrections, tests, amélioration UX, démo complète.

12. Démo cible de fin de V1

La première version doit permettre de démontrer le parcours suivant sans passer par des fichiers externes :

1. Créer un candidat.
2. Créer une entreprise.
3. Créer un besoin entreprise.
4. Proposer le candidat au besoin.
5. Marquer le candidat comme accepté.
6. Créer le dossier admission.
7. Ajouter les documents obligatoires.
8. Valider le dossier admission.
9. Créer le dossier contrat.
10. Marquer le contrat comme signé.
11. Créer le dossier OPCO.
12. Simuler un rejet puis une correction.
13. Afficher le dossier OPCO bloqué ou accepté dans le dashboard.
14. Afficher les tâches prioritaires et l'historique du dossier.

13. Conclusion

La méthode recommandée est donc de travailler par petites features indépendantes, de merger régulièrement dans develop et de corriger les conflits immédiatement. Cette organisation permet à deux développeurs juniors d'avancer sans se perdre dans un ERP trop vaste.

Le premier objectif n'est pas de livrer tout le cahier des charges, mais de rendre solide le parcours principal : candidat -> entreprise -> besoin -> matching -> admission -> contrat -> OPCO -> dashboard.